MySQL Stored Procedures — Practice Dataset

1. Dataset

We'll use a simple **company employee database**.

`employees`

employee_id	name	department	salary	experience_years
101	Rahul	IT	60000	3
102	Priya	HR	45000	5
103	Amit	IT	75000	8
104	Neha	Sales	50000	2
105	Arjun	Sales	65000	6
106	Sneha	HR	55000	4
107	Vikram	IT	90000	10
108	Anjali	Sales	48000	1

```
CREATE DATABASE company_db;
USE company_db;

CREATE TABLE employees (
    employee_id INT PRIMARY KEY,
    name VARCHAR(50),
    department VARCHAR(30),
    salary DECIMAL(10,2),
    experience_years INT
);

INSERT INTO employees
(employee_id, name, department, salary, experience_years)
VALUES
(101, 'Rahul', 'IT', 60000, 3),
(102, 'Priya', 'HR', 45000, 5),
```

```
(103, 'Amit', 'IT', 75000, 8),
(104, 'Neha', 'Sales', 50000, 2),
(105, 'Arjun', 'Sales', 65000, 6),
(106, 'Sneha', 'HR', 55000, 4),
(107, 'Vikram', 'IT', 90000, 10),
(108, 'Anjali', 'Sales', 48000, 1);
```

---

2. First: Why Do We Need Procedures?

Suppose your application repeatedly needs to find an employee.

Without a procedure, every application/program has to send:

```
SELECT *
FROM employees
WHERE employee_id = 103;
```

Another program might need:

```
SELECT *
FROM employees
WHERE employee_id = 107;
```

And another:

```
SELECT *
FROM employees
WHERE employee_id = 105;
```

The **logic is identical**. Only the employee ID changes.

Instead, create the procedure once:

```
DELIMITER //

CREATE PROCEDURE GetEmployee(IN emp_id INT)
BEGIN
    SELECT *
    FROM employees
```

```
    WHERE employee_id = emp_id;
END //

DELIMITER ;
```

Now:

```
CALL GetEmployee(103);
CALL GetEmployee(107);
CALL GetEmployee(105);
```

The important idea

> **A procedure allows you to store frequently used SQL logic once and reuse it with different inputs.**

---

3. Example: Giving Everyone in a Department a Raise

Imagine HR frequently needs to give employees a percentage raise.

Without a procedure, someone might repeatedly write:

```
UPDATE employees
SET salary = salary * 1.10
WHERE department = 'IT';
```

Next month:

```
UPDATE employees
SET salary = salary * 1.05
WHERE department = 'Sales';
```

Then:

```
UPDATE employees
SET salary = salary * 1.08
```

```
WHERE department = 'HR';
```

The logic is repetitive.

Create a procedure:

```
DELIMITER //

CREATE PROCEDURE GiveRaise(
    IN dept_name VARCHAR(30),
    IN raise_percent DECIMAL(5,2)
)
BEGIN
    UPDATE employees
    SET salary = salary + (salary * raise_percent / 100)
    WHERE department = dept_name;
END //

DELIMITER ;
```

Now:

```
CALL GiveRaise('IT', 10);
CALL GiveRaise('Sales', 5);
CALL GiveRaise('HR', 8);
```

One procedure handles all three cases.

---

4. Example: More Than Just Reusing SQL

Procedures become even more useful when you combine **multiple SQL statements + variables + conditions**.

Suppose HR wants to check an employee's salary category.

Rules:

- Salary >= 80,000 → `Excellent`
- Salary >= 60,000 → `Good`
- Salary >= 50,000 → `Average`

- Otherwise → `Low`

Without a procedure, you might repeatedly write a query containing this logic.

With a procedure:

```
DELIMITER //

CREATE PROCEDURE CheckSalary(IN emp_id INT)
BEGIN
    DECLARE emp_salary DECIMAL(10,2);

    SELECT salary
    INTO emp_salary
    FROM employees
    WHERE employee_id = emp_id;

    IF emp_salary >= 80000 THEN
        SELECT 'Excellent' AS salary_category;
    ELSEIF emp_salary >= 60000 THEN
        SELECT 'Good' AS salary_category;
    ELSEIF emp_salary >= 50000 THEN
        SELECT 'Average' AS salary_category;
    ELSE
        SELECT 'Low' AS salary_category;
    END IF;
END //

DELIMITER ;
```

Then:

```
CALL CheckSalary(101);
CALL CheckSalary(102);
CALL CheckSalary(107);
```

This demonstrates why procedures aren't merely "shortcuts for SELECT."

They can contain:

Input → variables → SQL → conditions → output

---

5. Example: Getting Department Statistics

Suppose management repeatedly asks:

> How many employees are in IT and what is their average salary?

You could write:

```
SELECT
    COUNT(*) AS employee_count,
    AVG(salary) AS average_salary
FROM employees
WHERE department = 'IT';
```

But the department changes.

Create:

```
DELIMITER //

CREATE PROCEDURE DepartmentStats(IN dept_name VARCHAR(30))
BEGIN
    SELECT
        COUNT(*) AS employee_count,
        AVG(salary) AS average_salary,
        MAX(salary) AS highest_salary,
        MIN(salary) AS lowest_salary
    FROM employees
    WHERE department = dept_name;
END //

DELIMITER ;
```

Now:

```
CALL DepartmentStats('IT');
CALL DepartmentStats('HR');
CALL DepartmentStats('Sales');
```

---

6. Example: OUT Parameter

Suppose you don't want the procedure to simply display the result. You want to **return the number of employees** to another SQL statement.

```
DELIMITER //

CREATE PROCEDURE CountEmployees(
    IN dept_name VARCHAR(30),
    OUT total INT
)
BEGIN
    SELECT COUNT(*)
    INTO total
    FROM employees
    WHERE department = dept_name;
END //

DELIMITER ;
```

Use:

```
CALL CountEmployees('IT', @total);

SELECT @total;
```

`@total` now contains the result.

This is where you should start thinking:

> **IN = give something to the procedure**
> **OUT = get something from the procedure**

---

7. A More Realistic Example

Suppose the company wants to give a bonus based on experience.

Rules:

- Experience >= 10 → 20% bonus
- Experience >= 5 → 10% bonus
- Experience < 5 → 5% bonus

We can make a procedure:

```
DELIMITER //

CREATE PROCEDURE CalculateBonus(IN emp_id INT)
BEGIN
    DECLARE emp_salary DECIMAL(10,2);
    DECLARE exp INT;
    DECLARE bonus DECIMAL(10,2);

    SELECT salary, experience_years
    INTO emp_salary, exp
    FROM employees
    WHERE employee_id = emp_id;

    IF exp >= 10 THEN
        SET bonus = emp_salary * 0.20;
    ELSEIF exp >= 5 THEN
        SET bonus = emp_salary * 0.10;
    ELSE
        SET bonus = emp_salary * 0.05;
    END IF;

    SELECT
        emp_id AS employee_id,
        emp_salary AS salary,
        exp AS experience,
        bonus;
END //

DELIMITER ;
```

Try:

```
CALL CalculateBonus(101);
CALL CalculateBonus(105);
CALL CalculateBonus(107);
```

This is a **very good exam-style procedure** because it combines:

- `IN`
- `DECLARE`
- `SELECT INTO`
- `SET`
- `IF / ELSEIF / ELSE`
- calculations
- `SELECT` output

---

8. Your Practice Questions

Try these **without looking at solutions first**.

Level 1 — Basic `IN`

Q1

Create a procedure called `GetEmployee`.

It should accept an employee ID and display:

- employee ID
- name
- department
- salary
- experience

Example:

```
CALL GetEmployee(105);
```

Q2

Create a procedure called `GetDepartmentEmployees`.

It should accept a department name and display all employees belonging to that department.

Example:

```
CALL GetDepartmentEmployees('IT');
```

Q3

Create a procedure called `GetHighEarners`.

It should accept a salary amount and display all employees whose salary is greater than that amount.

Example:

```
CALL GetHighEarners(60000);
```

Level 2 — Variables and Conditions

Q4

Create a procedure called `SalaryCategory`.

Input:

```
employee_id
```

Rules:

- salary >= 80000 → `Excellent`
- salary >= 60000 → `Good`
- salary >= 50000 → `Average`
- otherwise → `Low`

Example:

```
CALL SalaryCategory(107);
```

Q5

Create a procedure called `ExperienceLevel`.

Rules:

- experience >= 10 → `Expert`
- experience >= 5 → `Experienced`
- experience >= 2 → `Intermediate`
- otherwise → `Fresher`

Example:

```
CALL ExperienceLevel(105);
```

Q6

Create a procedure called `CalculateBonus` .

Input:

```
employee_id
```

Rules:

- experience >= 10 → 20% bonus
- experience >= 5 → 10% bonus
- otherwise → 5% bonus

Display:

```
employee_id
salary
bonus_percentage
bonus_amount
```

Level 3 — `OUT` Parameters

Q7

Create a procedure called `GetEmployeeCount` .

Input:

```
department
```

Output:

```
employee_count
```

Expected usage:

```
CALL GetEmployeeCount('IT', @count);
SELECT @count;
```

Q8

Create a procedure called `GetAverageSalary`.

Input:

```
department
```

Output:

```
average_salary
```

Expected:

```
CALL GetAverageSalary('Sales', @avg);
SELECT @avg;
```

Q9

Create a procedure called `GetHighestSalary`.

Input:

```
department
```

Output the highest salary in that department using an `OUT` parameter.

Level 4 — `UPDATE` Procedures

Q10

Create a procedure called `IncreaseSalary`.

Inputs:

```
employee_id
percentage
```

It should increase the employee's salary by the specified percentage.

Example:

```
CALL IncreaseSalary(101, 10);
```

Rahul's salary should increase from `60000` to `66000`.

Q11

Create a procedure called `DepartmentRaise`.

Inputs:

```
department
percentage
```

It should increase the salaries of **all employees** in that department.

Example:

```
CALL DepartmentRaise('IT', 10);
```

Level 5 — Challenge Questions 🔥

Q12

Create a procedure called `EmployeeReport`.

Input:

```
employee_id
```

The procedure should display:

- employee name
- department
- salary
- experience
- salary category
- experience category

You will need:

- `DECLARE`
- `SELECT INTO`
- `IF / ELSEIF`
- multiple variables
- final `SELECT`

Q13

Create a procedure called `DepartmentReport`.

Input:

```
department
```

It should display:

- number of employees

- average salary
- highest salary
- lowest salary
- total salary paid by the department

Q14 — Very Good Exam Question ⭐

Create a procedure called `GiveBonus`.

Input:

```
employee_id
```

Rules:

```
Experience >= 10  → 20% bonus
Experience >= 5   → 10% bonus
Experience < 5    → 5% bonus
```

The procedure should **return the bonus amount using an OUT parameter**.

Expected usage:

```
CALL GiveBonus(107, @bonus);

SELECT @bonus;
```

---

9. One Key Pattern to Memorize

Most beginner procedures follow this structure:

```
DELIMITER //

CREATE PROCEDURE ProcedureName(
    IN input_value datatype,
    OUT output_value datatype
)
BEGIN
```

```
    DECLARE variable datatype;

    SELECT ...
    INTO variable
    FROM ...
    WHERE ...;

    IF condition THEN
        SET variable = ...;
    ELSE
        SET variable = ...;
    END IF;

    SET output_value = variable;

END //

DELIMITER ;
```

If you can comfortably understand and write this pattern, you're ready for most **beginner-to-intermediate MySQL procedure questions**.